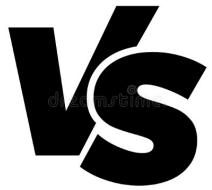
Étude comparative de performance entre Cypher 25 et SQL







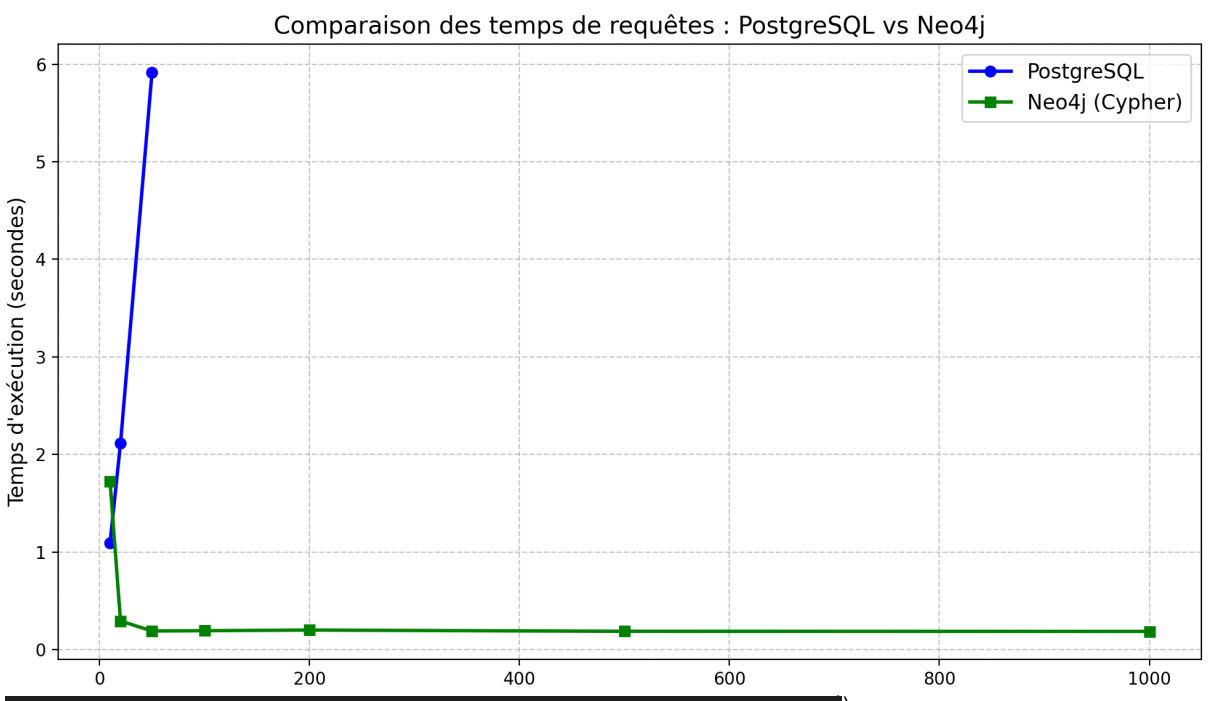
PROJET LONG – MASTER 1 INFORMATIQUE

29/05/2026

JIN CHRISTOPHE – M1 LP

PAN ALEXIS – M1 DATA

Introduction




```
Lancement test PostgreSQL vs Cypher
==================================================

Test avec budget max = 10
Test Cypher (RAM)... 1.723 secondes
Test PostgreSQL (Disque)... 1.094 secondes
------------------------------
Test avec budget max = 20
Test Cypher (RAM)... 0.293 secondes
Test PostgreSQL (Disque)... 2.116 secondes
------------------------------
Test avec budget max = 50
Test Cypher (RAM)... 0.191 secondes
Test PostgreSQL (Disque)... 5.919 secondes
------------------------------
Test avec budget max = 100
Test Cypher (RAM)... 0.194 secondes
Test PostgreSQL (Disque)... CRASH: canceling statement due to statement timeout

------------------------------
Test avec budget max = 200
Test Cypher (RAM)... 0.200 secondes
Test PostgreSQL (Disque)... CRASH: canceling statement due to statement timeout

------------------------------
Test avec budget max = 500
Test Cypher (RAM)... 0.188 secondes
Test PostgreSQL (Disque)... CRASH: canceling statement due to statement timeout

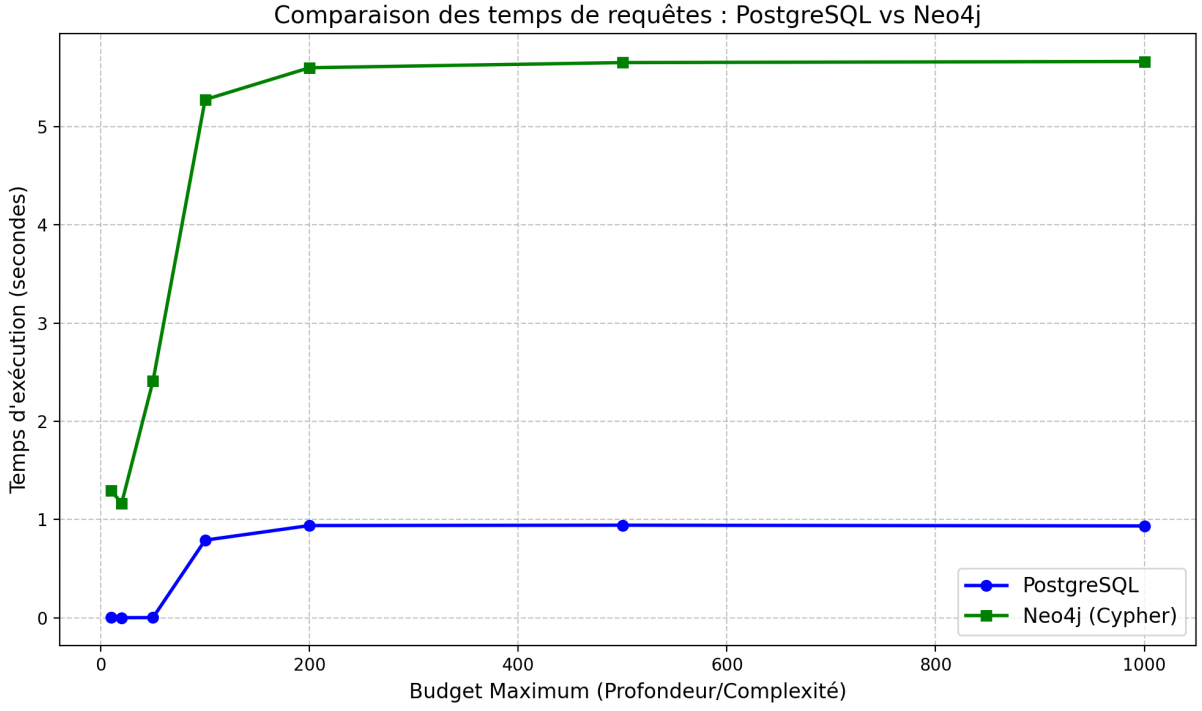
------------------------------
Test avec budget max = 1000
Test Cypher (RAM)... 0.187 secondes
Test PostgreSQL (Disque)... CRASH: canceling statement due to statement timeout

------------------------------
```

Graphe avec Cycle

```
==================================================
Lancement test PostgreSQL vs Cypher
==================================================

Test avec budget max = 10
Test Cypher (RAM)... 1.293 secondes
Test PostgreSQL (Disque)... 0.003 secondes
------------------------------
Test avec budget max = 20
Test Cypher (RAM)... 1.167 secondes
Test PostgreSQL (Disque)... 0.002 secondes
------------------------------
Test avec budget max = 50
Test Cypher (RAM)... 2.410 secondes
Test PostgreSQL (Disque)... 0.003 secondes
------------------------------
Test avec budget max = 100
Test Cypher (RAM)... 5.275 secondes
Test PostgreSQL (Disque)... 0.791 secondes
------------------------------
Test avec budget max = 200
Test Cypher (RAM)... 5.600 secondes
Test PostgreSQL (Disque)... 0.940 secondes
------------------------------
Test avec budget max = 500
Test Cypher (RAM)... 5.652 secondes
Test PostgreSQL (Disque)... 0.944 secondes
------------------------------
Test avec budget max = 1000
Test Cypher (RAM)... 5.663 secondes
Test PostgreSQL (Disque)... 0.936 secondes
------------------------------
```



Graphe sans Cycle

Architecture

```python
def new_generation(width, depths):
    with open('noeuds.csv', 'w', newline='', encoding='utf-8') as fn, open('aretes.csv', 'w', newline='', encoding='utf-8') as fa:
        # Initialisation de la tete de lecture
        wn = csv.writer(fn)
        wa = csv.writer(fa)
        wn.writerow(['id', 'profit'])
        wa.writerow(['source', 'target', 'cost'])

        # Noeud de départ
        curr = "Depart"
        wn.writerow([curr, 0])

        for i in range(depths):
            next = f"Opt_{i}"
            if i != depths // 2:
                wn.writerow([next, random.randint(1, 3)])
            else:
                # Donner une valeur absurde pour être sûr que le chemin est le plus opti
                wn.writerow([next, width * depths])
            wa.writerow([curr, next, 1])
            curr = next

        wn.writerow(['Fin', random.randint(1, 3)])
        wa.writerow([curr, 'Fin', 1])

        for i in range(width):
            recursive_bait(i, depths, curr, wn, wa)


trap_node = 0
nodes = []


def recursive_bait(branch, depths, prev, wn, wa):
    global trap_node, nodes
    rand = random.choice([1] * 90 + [2] * 10)
    for i in range(rand):
        next = f"Piege_{trap_node}"
        trap_node += 1

        if next not in nodes:
            wn.writerow([next, random.randint(50, 100)])
            nodes.append(next)
        wa.writerow([prev, next, 1])

        if depths == 0:
            wa.writerow([next, 'Fin', 1])
        else:
            recursive_bait(branch, depths - 1, next, wn, wa)
```

```python
# Fonction qui génére le graphe massif en séparant les branches et les noeuds
def generer_graphe(nb_noeuds_optimaux, nb_branches_pieges):

    # On ouvre les fichiers directement pour écrire ligne par ligne
    with open('noeuds.csv', 'w', newline='', encoding='utf-8') as fn, open('aretes.csv', 'w', newline='', encoding='utf-8') as fa:

        #Initialisation de la tete de lecture
        writer_n = csv.writer(fn)
        writer_a = csv.writer(fa)
        writer_n.writerow(['id', 'profit'])
        writer_a.writerow(['source', 'target', 'cost'])

        # On s'occupe de faire d'abord le chemin optimal (pas rentable en cours terme)
        writer_n.writerow(['Depart', 0])
        noeud_courant = 'Depart'

        for i in range(1, nb_noeuds_optimaux + 1):

            noeud_suivant = f'Opt_{i}'
            writer_n.writerow([noeud_suivant, random.randint(1, 3)])
            writer_a.writerow([noeud_courant, noeud_suivant, 1])
            noeud_courant = noeud_suivant

        writer_n.writerow(['Fin', 50])
        writer_a.writerow([noeud_courant, 'Fin', 1])

        # On s'occupe fde faire les pieges
        for i in range(nb_branches_pieges):
            piege_a = f'Piege_A_{i}'
            piege_b = f'Piege_B_{i}'

            # Des profits immédiats monstrueux pour aveugler l'optimiseur
            writer_n.writerow([piege_a, random.randint(50, 100)])
            writer_n.writerow([piege_b, random.randint(10, 20)])

            # On les connecte au départ
            writer_a.writerow(['Depart', piege_a, 1])

            # On les multiblie
            writer_a.writerow([piege_a, piege_b, 1])
            writer_a.writerow([piege_b, piege_a, 2])

if __name__ == "__main__":
    print("Génération des csv du graphe. ")

    generer_graphe(nb_noeuds_optimaux=100, nb_branches_pieges=50000)

    print("Fichiers noeuds.csv et aretes.csv sont générés. ")
```

Conception graphes

Injections : Cypher PostgreSQL

```python
# Connexion
load_dotenv()
URI = os.getenv("NEO4J_URI")
USERNAME = os.getenv("NEO4J_USER")
PASSWORD = os.getenv("NEO4J_PASSWORD")


print("Connexion a :", URI)
driver = GraphDatabase.driver(URI, auth=(USERNAME, PASSWORD))


# Fonction qui vide toute la bdd
def reset_database(session):

    # L'exécution doit se faire directement sur la session en mode "auto-commit"
    session.run("""
        MATCH (n)
        CALL { WITH n DETACH DELETE n } IN TRANSACTIONS OF 10000 ROWS
    """)

    print("Base de données Neo4j réinitialisée.")


# Insertion des noeuds
def ingest_nodes_batch(tx, batch):
    query = """
    UNWIND $batch AS row
    CREATE (:Node {id: row.id, profit: toInteger(row.profit)})
    """
    tx.run(query, batch=batch)


# Insertion des aretes entre les noeuds
def ingest_edges_batch(tx, batch):
    query = """
    UNWIND $batch AS row
    MATCH (source:Node {id: row.source})
    MATCH (target:Node {id: row.target})
    CREATE (source)-[:CONNECTED {cost: toInteger(row.cost)}]->(target)
    """
    tx.run(query, batch=batch)


# Fonction de lecture des csv et découpage en paquet
def process_csv_in_batches(session, filepath, batch_size, ingestion_function):

    # Ouverture du csv
    with open(filepath, 'r', encoding='utf-8') as file:
        reader = csv.DictReader(file)
        batch = []
        total_processed = 0

        # Insertion des lignes
        for row in reader:
            batch.append(row)
            if len(batch) >= batch_size:
                session.execute_write(ingestion_function, batch)
                total_processed += len(batch)
                batch = []

        # On insère le reste s'il y a un paquet incomplet à la fin
        if batch:
            session.execute_write(ingestion_function, batch)
            total_processed += len(batch)

# Execution principale
with driver.session() as session:

    reset_database(session)
    # Taille des paquets
    BATCH_SIZE = 10000

    # Injection des noeuds
    print("Injection des noeuds en cours. ")
    process_csv_in_batches(session, 'noeuds.csv', BATCH_SIZE, ingest_nodes_batch)

    # Création d'index
    session.run("CREATE INDEX node_id_index IF NOT EXISTS FOR (n:Node) ON (n.id)")
    session.run("CALL db.awaitIndexes()")
    print("Index de recherche créés. ")

    # Ajour aretes
    print("Injection des aretes en cours. ")
    process_csv_in_batches(session, 'aretes.csv', BATCH_SIZE, ingest_edges_batch)


print("Injection Neo4j terminée ! ")
driver.close()
```

```python
# Load .env file
load_dotenv()
# Read variables
PG_HOST = os.getenv("PG_HOST", "localhost")
PG_PORT = os.getenv("PG_PORT", "5432")
PG_DB = os.getenv("PG_DB", "postgres")
PG_USER = os.getenv("PG_USER", "postgres")
PG_PASSWORD = os.getenv("PG_PASSWORD")


# Optional: quick debug
print("Connecting to PostgreSQL at:", PG_HOST)
print("User:", PG_USER)


# Create driver (pour se connecter a la base de donnée de PostgreSQL)
conn = psycopg2.connect(
    host=PG_HOST,
    port=PG_PORT,
    dbname=PG_DB,
    user=PG_USER,
    password=PG_PASSWORD
)


# Pour éviter de faire conn.commit() à chaque fois
conn.autocommit = True
cursor = conn.cursor()


# Fonction qui vide toute la bdd
def reset_database(cur):
    cur.execute("DROP TABLE IF EXISTS aretes;")
    cur.execute("DROP TABLE IF EXISTS noeuds;")

    cur.execute("""
        CREATE TABLE noeuds (
            id VARCHAR(255) PRIMARY KEY,
            profit INTEGER
        );
    """)
    cur.execute("""
        CREATE TABLE aretes (
            source VARCHAR(255),
            target VARCHAR(255),
            cost INTEGER
        );
    """)
    print("Tables 'noeuds' et 'aretes' réinitialisées.")


# Pour compter le nombre de noeud dans la bdd (un petit check pour verifier qu'on a bien tout)
def get_nodes_count(cur):
    cur.execute("SELECT COUNT(*) FROM noeuds;")
    return cur.fetchone()[0]


def write_data(cur):
    # Write data (On utilise copy_from, la méthode la plus rapide pour PostgreSQL)
    print("Injection des noeuds en cours. ")

    with open('noeuds.csv', 'r', encoding='utf-8') as f:
        # On ignore la première ligne du CSV (les en-têtes id,profit)
        next(f)
        cur.copy_from(f, 'noeuds', sep=',')
    print("Noeuds insérés! ")

    print("Injection des aretes en cours. ")
    with open('aretes.csv', 'r', encoding='utf-8') as f:
        # On ignore la première ligne du CSV (les en-têtes source,target,cost)
        next(f)
        cur.copy_from(f, 'aretes', sep=',')
    print("Arêtes insérées!")

    # Création des index
    cur.execute("CREATE INDEX idx_aretes_source ON aretes(source);")
    cur.execute("CREATE INDEX idx_aretes_target ON aretes(target);")
    print("Index de recherche créés.")

# Execution du script
try:
    reset_database(cursor)
    write_data(cursor)

    nodes_count = get_nodes_count(cursor)
    print("nodes in database:", nodes_count)

except Exception as e:
    print("Une erreur est survenue :", e)

finally:
    cursor.close()
    conn.close()
    print("Connexion a PostgreSQL terminé.")
```

```python
# Fonction de test sur Cypher
def test_neo4j(budget):
    # Parcours des chemins en utilisant allreduce()
    query = """
    MATCH p = (d:Node {id: 'Depart'})-[:CONNECTED*]->(end:Node)

    // allReduce(accumulateur, variable de boucle | fonction d'accumulation, condition de survie)
    WHERE allReduce(cout = 0, r IN relationships(p) | cout + r.cost, cout <= $budget)

    RETURN [n IN nodes(p) | n.id] AS chemin, reduce(c = 0, r IN relationships(p)
    | c + r.cost) AS cout_total, reduce(pr = d.profit, n IN nodes(p)[1..] | pr + n.profit) AS profit_total
    ORDER BY profit_total DESC
    LIMIT 1
    """

    # Code pas du tout optimal pour faire cracher
    # query = """
    #    MATCH p = (d:Node {id: 'Depart'})-[:CONNECTED*]->(end:Node)
    #    WITH p,
    #        reduce(c = 0, r IN relationships(p) | c + r.cost) AS cout_total,
    #        reduce(pr = toInteger(d.profit), n IN nodes(p)[1..] | pr + toInteger(n.profit)) AS profit_total
    #
    #    RETURN [n IN nodes(p) | n.id] AS chemin, cout_total, profit_total
    #    ORDER BY profit_total DESC
    #    LIMIT 1000
    #"""

    # Timeur
    start_time = time.time()
    # Il faudra nous meme brider la RAM que le moteur JAVA a le droit d'utiliser, on ouvre le fichier neo4j.conf,
    # puis on modifie ces 2 lignes | server.memory.heap.initial_size=512m server.memory.heap.max_size=1G |
    try:
        with neo4j_driver.session() as session:
            session.run(query, budget=budget).single()

        elapsed = time.time() - start_time
        return elapsed  # Retourne un float pour le graphique

    except Exception as e:
        print(f"CRASH: {e}")
        return None  # Retourne None en cas de crash
```

Benchmark

```python
# Fonction de test sur PostgreSQL
def test_postgresql(budget):
    # Boucle WITH RECURSIVE qui parcours les chemins
    query = """
    WITH RECURSIVE exploration AS (

        -- On force le type en ::TEXT pour éviter l'erreur de typage
        SELECT id AS noeud_actuel, ARRAY[id::TEXT] AS chemin, 0 AS cout_total, profit AS profit_total
        FROM noeuds WHERE id = 'Depart'

        UNION ALL

        -- On force aussi la cible en ::TEXT au moment de l'ajouter
        SELECT a.target, e.chemin || a.target::TEXT, e.cout_total + a.cost, e.profit_total + n.profit
        FROM exploration e
        JOIN aretes a ON e.noeud_actuel = a.source
        JOIN noeuds n ON a.target = n.id
        WHERE (e.cout_total + a.cost) <= %(budget)s
    )

    -- On prend le chemin qui a généré le plus de profit
    SELECT chemin, cout_total, profit_total
    FROM exploration
    ORDER BY profit_total DESC
    LIMIT 1;
    """

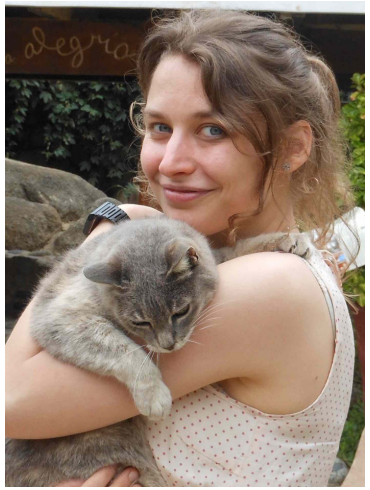
    # Timeur
    start_time = time.time()

    # Étant donné qu'on a pas vraiment de brider postgres, nous allons nous meme le faire crasher manuellement
    try:
        with pg_conn.cursor() as cur:
            # Un timeout dans le pire cas
            cur.execute("SET statement_timeout = '10s';")
            cur.execute(query, {'budget': budget})
            resultat = cur.fetchone()

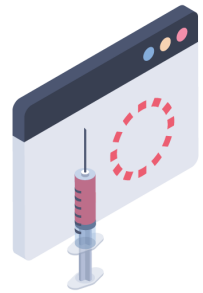
        elapsed = time.time() - start_time
        return elapsed  # Retourne un float pour le graphique

    except Exception as e:
        pg_conn.rollback()
        print(f"CRASH: {e}")
        return None  # Retourne None en cas de crash
```









CODE INJECTION





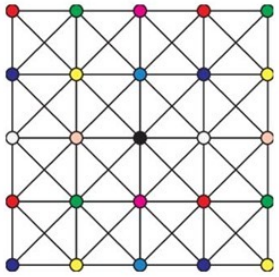
PostgreSQL Support Timeline

Release	Release Date	End of Life Date
PostgreSQL 13	24 September, 2020	13 November, 2025
PostgreSQL 14	30 September, 2021	12 November, 2026
PostgreSQL 15	13 October, 2022	11 November, 2027
PostgreSQL 16	14 September, 2023	9 November, 2028
PostgreSQL 17	26 September, 2024	8 November, 2029
PostgreSQL 18	25 September, 2025	14 November, 2030

Review our Neo4j Support Terms for reference.

Release	Release Date	Hotfixes until	Compatible Driver Versions
2026.04.0	April 23, 2026	Release of 2026.05	6.x, 5.x, 4.4 (limited)
2026.03.1	April 2, 2026	Release of 2026.04	6.x, 5.x, 4.4 (limited)
2026.02.3	March 20, 2026	Release of 2026.03	6.x, 5.x, 4.4 (limited)
2026.01.4	February 10, 2026	Release of 2026.02	6.x, 5.x, 4.4 (limited)
2025.12.1	January 16, 2026	Release of 2026.01	6.x, 5.x, 4.4 (limited)
2025.11.2	December 19, 2025	Release of 2025.12	6.x, 5.x, 4.4 (limited)
2025.10.1	October 30, 2025	Release of 2025.11	6.x, 5.x, 4.4 (limited)
2025.09.0	September 29, 2025	Release of 2025.10	6.x, 5.x, 4.4 (limited)
2025.08.0	August 27, 2025	Release of 2025.09	6.x, 5.x, 4.4 (limited)
2025.07.0	July 31, 2025	Release of 2025.08	6.x, 5.x, 4.4 (limited)
2025.06.0	July 2, 2025	Release of 2025.07	6.x, 5.x, 4.4 (limited)
2025.05.0	June 3, 2025	Release of 2025.06	6.x, 5.x, 4.4 (limited)
2025.04.0	April 30, 2025	Release of 2025.05	6.x, 5.x, 4.4 (limited)
2025.03.0	March 27, 2025	Release of 2025.04	6.x, 5.x, 4.4 (limited)
2025.02.0	February 27, 2025	Release of 2025.03	6.x, 5.x, 4.4 (limited)
2025.01.0	February 06, 2025	Release of 2025.02	6.x, 5.x, 4.4 (limited)

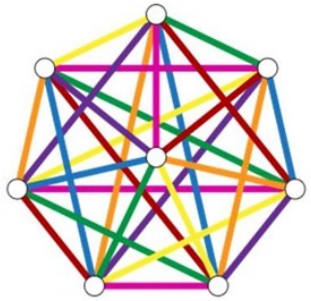
coloriage de graphes



nœuds

coloriage avec k couleurs : **NP-complet**
trouver le nombre minimum de couleurs : **NP-difficile**

allocation de fréquences radio télécoms



segments

coloriage avec k couleurs : **P**
coloriage optimum : **NP-difficile**

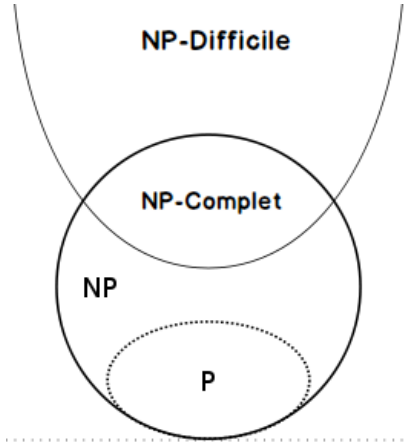
allocation de fréquences de fibres optiques multimodes



zones

déterminer si coloriage possible avec 3 couleurs : **NP-complet**
coloriage avec 4 couleurs : **P**

définition de zones de couverture d'antennes de réseaux télécoms
allocation de fréquences micro-ondes dans des réseaux de qubits supraconducteurs



$P \neq NP$

Quelles sont les difficultés technologiques et techniques rencontrées ?

Programmation

Génération d'un chemin optimal

```
    for i in range(depths):
        next = f"Opt_{i}"
        if i != depths // 2:
            wn.writerow([next, random.randint(1, 3)])
        else:
            # Donner une valeur absurde pour être sûr que le chemin est le plus opti
            wn.writerow([next, width * depths])
        wa.writerow([curr, next, 1])
        curr = next

    wn.writerow(['Fin', random.randint(1, 3)])
    wa.writerow([curr, 'Fin', 1])

    for i in range(width):
        recursive_bait(i, depths, curr, wn, wa)
```

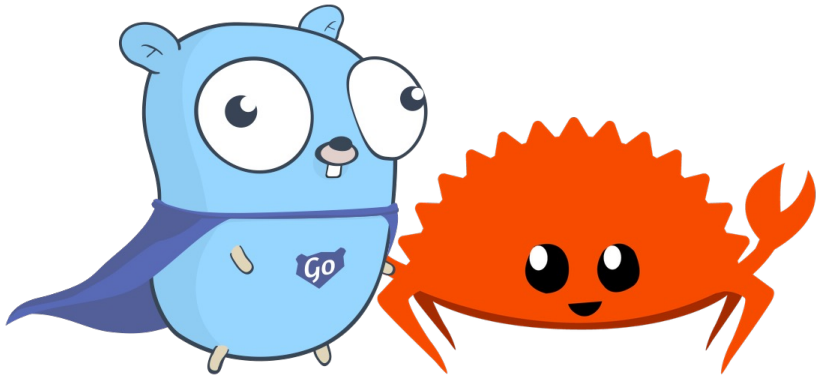
```
def recursive_bait(branch, depths, prev, wn, wa):
    global trap_node, nodes
    rand = random.choice([1] * 90 + [2] * 10)
    for i in range(rand):
        next = f"Piege_{trap_node}"
        trap_node += 1

        if next not in nodes:
            wn.writerow([next, random.randint(50, 100)])
            nodes.append(next)
        wa.writerow([prev, next, 1])

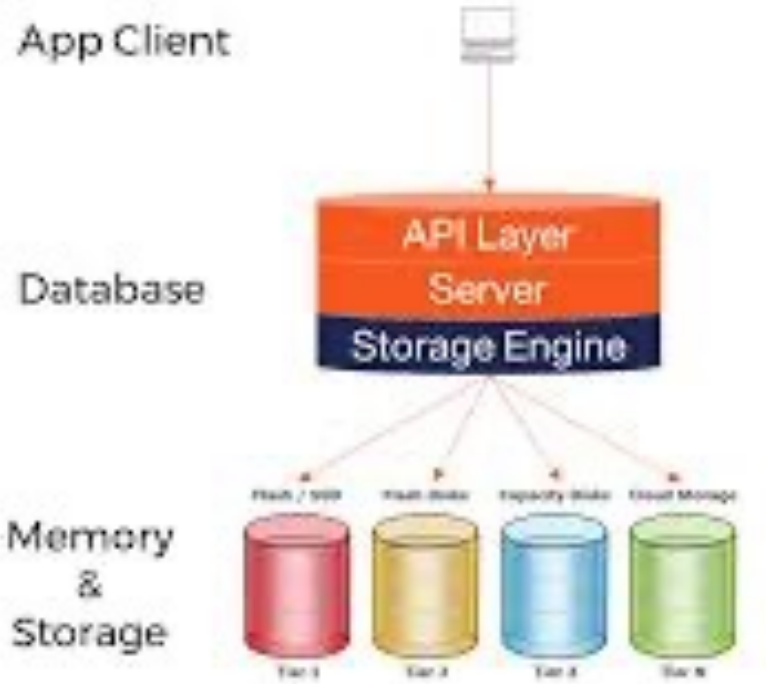
        if depths == 0:
            wa.writerow([next, 'Fin', 1])
        else:
            recursive_bait(branch, depths - 1, next, wn, wa)
```

Génération de chemins pièges qui se divisent et fusionnent




Go

Conclusion


App Client
Database
API Layer
Server
Storage Engine
Memory
&
Storage